

Laporan Tugas Besar 1

IF2211 Strategi Algoritma

Pemanfaatan *Algoritma Greedy* - Pembuatan Bot Permainan Battlecode 2025

Semester II Tahun Ajaran 2025/2026

IF 2024 - Jatinangor

Tim GitHub: the_Brothers



Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung



Kata Pengantar

Dokumen Laporan Tugas Besar 1 ini disusun oleh kelompok (the_Brothers) Strategi Algoritma dari mahasiswa Teknik Informatika angkatan 2024. Adapun susunan anggota kelompok kami secara detail sebagai berikut:

1. **Fazri** Arrashyi Putra (NIM: 13524**127**)
2. **Dika** Pramudya Nugraha (NIM: 13524**132**)
3. **Dzakwan** Muhammad Khairan Putra Purnama (NIM: 13524**145**)



Daftar Isi

Kata Pengantar.....	1
Daftar Isi.....	2
1. Deskripsi Tugas.....	3
2. Landasan Teori.....	6
2.1. Konsep Dasar Algoritma Greedy.....	6
2.2. Sifat Utama Algoritma Greedy.....	7
2.3. Elemen Pembentuk Algoritma.....	7
2.4. Relevansi Greedy pada Pengembangan Bot Battlecode.....	8
3. Aplikasi strategi greedy.....	8
4. Implementasi dan Pengujian.....	16
5. POST MORTEM Analisis.....	31
6. Kesimpulan dan Saran.....	32
7. Lampiran.....	33
8. Daftar Pustaka.....	35

1. Deskripsi Tugas

Tugas besar pertama pada mata kuliah IF2211 Strategi Algoritma berfokus pada penerapan algoritma greedy dalam sebuah permasalahan strategis yang dinamis. Permasalahan tersebut dikemas dalam bentuk permainan Battlecode 2025, yaitu sebuah permainan strategi berbasis kecerdasan buatan di mana setiap peserta harus mengembangkan bot yang mampu mengambil keputusan secara otomatis untuk memenangkan permainan. Dalam konteks tugas ini, mahasiswa tidak mengendalikan unit permainan secara langsung, melainkan merancang logika dan strategi melalui kode program yang kemudian dijalankan oleh unit unit robot di dalam permainan.



Gambar 1. Battle Code 2025.

Battlecode merupakan permainan strategi berbasis giliran yang berlangsung pada sebuah peta dua dimensi berbentuk grid. Setiap pertandingan melibatkan dua tim yang masing masing memiliki sejumlah robot dan menara. Robot robot tersebut akan menjalankan kode program yang sama secara independen pada setiap ronde permainan. Melalui program yang dibuat, robot dapat melakukan berbagai aksi seperti bergerak, menyerang, mewarnai wilayah, membangun menara, maupun berkomunikasi dengan unit sekutu lainnya. Dengan demikian, keberhasilan suatu tim dalam permainan sangat bergantung pada kualitas strategi dan algoritma yang diimplementasikan dalam bot yang dibuat.

Tujuan utama dari permainan ini adalah menguasai wilayah peta dengan cara mewarnai sebanyak mungkin petak yang tersedia. Sebuah tim dinyatakan menang apabila berhasil mewarnai lebih dari tujuh puluh persen dari total petak yang dapat diwarnai pada peta. Selain itu, kemenangan juga dapat diperoleh apabila suatu tim berhasil menghancurkan seluruh robot dan menara milik tim lawan. Permainan berlangsung hingga maksimal dua ribu ronde. Apabila hingga batas tersebut belum ada tim yang memenuhi kondisi kemenangan, maka pemenang

akan ditentukan berdasarkan beberapa kriteria tambahan seperti luas wilayah yang berhasil dikuasai, jumlah menara yang masih bertahan, jumlah sumber daya yang dimiliki, serta jumlah robot yang masih hidup. Setiap pertandingan berlangsung pada peta berbentuk grid persegi panjang dua dimensi dengan ukuran yang bervariasi antara dua puluh kali dua puluh hingga enam puluh kali enam puluh petak. Setiap posisi pada peta direpresentasikan menggunakan koordinat kartesius dengan titik asal berada pada bagian kiri bawah peta. Untuk menjaga keseimbangan permainan, peta yang digunakan bersifat simetris sehingga tidak memberikan keuntungan khusus bagi salah satu tim. Selain itu, beberapa petak pada peta dapat berisi tembok yang tidak dapat dilewati maupun diwarnai oleh robot. Keberadaan elemen tersebut menambah kompleksitas pergerakan dan strategi yang harus dipertimbangkan oleh bot yang dikembangkan. Dalam permainan ini terdapat beberapa komponen utama yang mempengaruhi jalannya permainan, yaitu robot, menara, serta berbagai sumber daya yang digunakan selama pertandingan. Robot merupakan unit utama yang melakukan berbagai aksi di dalam permainan. Setiap robot memiliki kemampuan untuk bergerak, melakukan pengamatan terhadap lingkungan sekitar, menyerang target tertentu, serta melakukan koordinasi dengan unit lain melalui mekanisme komunikasi yang tersedia. Setiap robot juga memiliki atribut tertentu seperti jumlah hit points, kapasitas cat, biaya produksi, serta cooldown untuk melakukan aksi tertentu.

Terdapat **tiga jenis robot utama** dalam permainan ini. Jenis pertama adalah Soldier yang berperan sebagai unit dasar dengan kemampuan untuk mewarnai wilayah serta menyerang menara lawan. Jenis kedua adalah Splasher yang memiliki kemampuan menyerang dalam area tertentu sehingga dapat mewarnai beberapa petak sekaligus dalam satu aksi. Jenis ketiga adalah Mopper yang berfungsi untuk menghapus cat milik lawan serta mendistribusikan cat kepada robot sekutu lainnya. Perbedaan kemampuan tersebut menyebabkan setiap jenis robot memiliki peran strategis yang berbeda dalam permainan. Selain robot, terdapat pula menara yang berfungsi sebagai pusat produksi sumber daya dan titik spawn bagi robot baru. Menara dapat dibangun pada lokasi reruntuhan tertentu dengan cara menyelesaikan pola pewarnaan tertentu pada area di sekitarnya. Menara juga memiliki kemampuan untuk menyerang unit musuh serta menghasilkan sumber daya secara pasif pada setiap ronde permainan. Dalam permainan ini terdapat beberapa jenis menara dengan fungsi yang berbeda, seperti menara penghasil cat, menara penghasil chips, serta menara pertahanan yang memberikan peningkatan kekuatan serangan bagi menara sekutu lainnya. Permainan Battlecode menggunakan dua jenis sumber daya utama yaitu cat dan chips. Cat merupakan sumber daya yang digunakan untuk mewarnai wilayah serta menjalankan berbagai kemampuan robot seperti serangan dan penandaan wilayah. Setiap robot maupun menara memiliki kapasitas penyimpanan cat yang terbatas sehingga pengelolaan sumber daya ini menjadi salah satu aspek penting dalam strategi permainan. Sementara itu, chips berfungsi sebagai mata uang dalam permainan yang digunakan untuk memproduksi robot baru maupun melakukan pengembangan terhadap menara yang dimiliki.

Selain mekanisme dasar tersebut, permainan juga menyediakan fitur tambahan berupa pola sumber daya khusus yang dapat memberikan keuntungan bagi tim yang berhasil menyelesaikannya. Pola ini harus diwarnai dengan konfigurasi tertentu pada area peta tertentu.

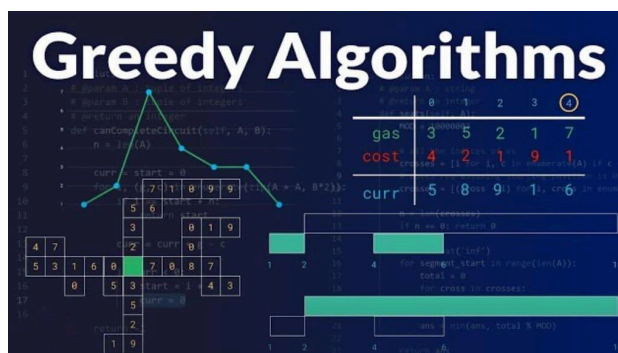
Apabila pola tersebut berhasil dipertahankan dalam kondisi utuh selama beberapa ronde, maka seluruh menara penghasil sumber daya milik tim tersebut akan memperoleh peningkatan produksi setiap ronde. Dalam menjalankan strategi permainan, robot robot dalam satu tim tidak dapat berbagi variabel secara langsung karena setiap robot menjalankan instansi program yang terpisah. Oleh karena itu, koordinasi antar robot harus dilakukan melalui mekanisme komunikasi yang tersedia dalam permainan. Komunikasi dapat dilakukan melalui pengiriman pesan singkat antar unit maupun melalui penanda yang ditempatkan pada peta untuk memberikan informasi tertentu kepada robot sekutu lainnya.

Pada tugas besar ini, setiap kelompok diminta untuk mengembangkan tiga buah bot menggunakan bahasa pemrograman Java. Ketiga bot tersebut harus mengimplementasikan pendekatan algoritma greedy dalam proses pengambilan keputusan. Satu bot akan digunakan sebagai bot utama, sedangkan dua bot lainnya berfungsi sebagai alternatif strategi yang berbeda. Setiap strategi greedy yang dirancang harus menggunakan fungsi heuristik yang berbeda sehingga menghasilkan pendekatan pengambilan keputusan yang berbeda pula. Dalam proses pengembangannya, strategi greedy yang digunakan harus dikaitkan dengan fungsi objektif permainan, yaitu memaksimalkan penguasaan wilayah peta.

Dengan demikian, setiap bot harus mampu menentukan aksi terbaik pada setiap kondisi permainan berdasarkan informasi yang tersedia pada saat itu. Selain itu, implementasi program juga harus memperhatikan berbagai batasan sistem yang ditetapkan oleh permainan, seperti batas bytecode yang dapat digunakan pada setiap giliran, penggunaan memori yang terbatas, serta waktu eksekusi program secara keseluruhan. Melalui tugas besar ini diharapkan mahasiswa dapat memahami secara lebih mendalam bagaimana konsep algoritma greedy dapat diterapkan pada permasalahan nyata yang bersifat dinamis dan kompleks. Selain itu, tugas ini juga melatih kemampuan mahasiswa dalam merancang strategi algoritmik, mengimplementasikan solusi dalam bentuk program, serta melakukan analisis terhadap performa solusi yang dihasilkan.

2. Landasan Teori

2.1. Konsep Dasar Algoritma Greedy



Gambar 2. Algoritma Greedy.

Algoritma *greedy* adalah metode yang paling populer dan sederhana untuk memecahkan berbagai persoalan optimasi^[1]. Secara umum, persoalan optimasi bertujuan untuk mencari solusi terbaik, yang terbagi menjadi dua fokus utama:

- **Maksimasi (Maximization):** Mencari nilai tertinggi, seperti memaksimalkan keuntungan atau jumlah area yang dikuasai.
- **Minimasi (Minimization):** Mencari nilai terendah, seperti meminimalkan biaya, jarak tempuh, atau waktu eksekusi^[1].

Prinsip utama dari algoritma ini dapat dirangkum dalam kalimat "**take what you can get now!**"^[1]. Dalam pendekatannya, algoritma *greedy* membangun solusi secara bertahap. Pada setiap langkahnya, algoritma ini akan mengambil keputusan terbaik pada saat itu juga (disebut sebagai optimum lokal). Keputusan yang sudah diambil bersifat mutlak dan tidak bisa dievaluasi ulang di masa depan^[1]. Pendekatan ini berharap bahwa kumpulan keputusan optimum lokal tersebut pada akhirnya akan membentuk solusi terbaik secara keseluruhan (optimum global)^[1].

2.2. Sifat Utama Algoritma Greedy

Agar sebuah masalah dapat diselesaikan dengan baik menggunakan pendekatan greedy, masalah tersebut umumnya harus memiliki dua sifat dasar^[1]:

- **Greedy Choice Property:** Solusi keseluruhan yang optimal dapat dibangun dengan cara merangkai pilihan-pilihan lokal terbaik.
- **Optimal Substructure:** Solusi optimal dari sebuah masalah besar sebenarnya merupakan gabungan dari solusi-solusi optimal pada masalah-masalah pecahannya yang lebih kecil.

Kedua sifat ini memastikan bahwa keputusan instan yang diambil pada setiap langkah tidak akan merusak peluang untuk mendapatkan hasil akhir yang baik, yang menjadi dasar keberhasilan algoritma ini pada berbagai persoalan klasik seperti pencarian rute terpendek atau pemampatan data^{[2][3]}.

2.3. Elemen Pembentuk Algoritma

Secara teoritis, rancangan program yang menggunakan algoritma greedy tersusun atas enam elemen atau fungsi utama^[1]:

- **Himpunan Kandidat:** Daftar elemen mentah yang tersedia untuk dipilih pada setiap gilirannya (misalnya daftar target petak atau musuh terdekat).
- **Himpunan Solusi:** Kumpulan elemen dari kandidat yang sudah divalidasi dan dipilih.
- **Fungsi Seleksi:** Aturan atau logika (heuristik) untuk memilih kandidat yang paling menguntungkan saat ini.
- **Fungsi Kelayakan:** Pengecekan untuk memastikan kandidat yang dipilih sah dan tidak melanggar aturan sistem.
- **Fungsi Obyektif:** Tujuan akhir yang ingin dicapai (misalnya total skor atau area).
- **Fungsi Solusi:** Evaluator yang menentukan apakah proses pencarian sudah selesai atau masih harus dilanjutkan.

2.4. Relevansi Greedy pada Pengembangan Bot Battlecode

Battlecode adalah permainan strategi waktu nyata (Real-Time Strategy) di mana pemain harus memprogram sebuah bot otonom untuk saling bertanding^[4]. Tujuan utamanya adalah menguasai lebih dari 70% area permainan atau menghancurkan seluruh pasukan lawan^[4].

Dalam konteks teori komputasi, pengembangan bot ini menghadapi satu tantangan utama: keterbatasan sumber daya komputasi (memori/waktu) pada setiap giliran (*turn*)^[4]. Di sinilah algoritma *greedy* menjadi landasan teori yang sangat krusial.

Ketimbang menggunakan algoritma pencarian kompleks yang mensimulasikan berbagai kemungkinan masa depan (yang akan memakan banyak komputasi dan berisiko *time-out*), bot dapat dirancang menggunakan fungsi seleksi *greedy*. Bot cukup melihat kondisi peta saat ini dan langsung mengambil tindakan paling optimal di sekitarnya secara instan seperti bergerak ke petak kosong terdekat atau menyerang musuh dengan *health point* terendah. Efisiensi inilah yang membuat *greedy* menjadi standar yang andal untuk sistem yang menuntut pengambilan keputusan cepat dengan sumber daya terbatas.

Footnote Information:

-
- [1]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: Algoritma Greedy (Bagian 1)* [Online]. Available: 04-Algoritma-Greedy-(2026)-Bag1.pdf
[2]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: Algoritma Greedy (Bagian 2)* [Online]. Available: 05-Algoritma-Greedy-(2026)-Bag2.pdf
[3]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: Algoritma Greedy (Bagian 3)* [Online]. Available: 06-Algoritma-Greedy-(2026)-Bag3.pdf
[4]. **Program Studi Teknik Informatika ITB**, *Tugas Besar 1 IF2211 Strategi Algoritma Semester II tahun 2025/2026* [Online]. Available: Copy of Spesifikasi Tugas Besar 1 - IF2211 - 2025_2026.pdf
-

3. Aplikasi strategi greedy

3.1. Bot Utama

Bot utama dirancang untuk menggabungkan beberapa strategi penguasaan wilayah secara simultan, yaitu pewarnaan tile, pembangunan menara dari ruin, serta pengendalian area menggunakan robot *Splasher* dan *Mopper*. Berbeda dengan bot alternatif yang hanya berfokus pada satu aspek permainan, bot utama menerapkan pendekatan yang lebih seimbang antara ekspansi wilayah dan penguatan ekonomi melalui pembangunan menara. Dengan pendekatan ini, setiap robot memilih aksi yang memberikan keuntungan lokal terbesar berdasarkan kondisi lingkungan yang dapat diamati pada saat itu.

3.1.1. Greedy Pemilihan Target Pembangunan Menara

Tujuan strategi ini adalah memilih *ruin* yang paling efektif untuk dikonversi menjadi menara. Soldier memindai seluruh tile di sekitar robot untuk mendeteksi keberadaan ruin yang masih tersedia.

- **Himpunan Kandidat:** seluruh tile yang memiliki atribut ruin di sekitar posisi robot.
- **Himpunan Solusi:** satu ruin terbaik yang dipilih sebagai target pembangunan menara.
- **Fungsi Seleksi:**
 1. Memeriksa apakah ruin sudah memiliki menara.
 2. Mengevaluasi jumlah Soldier sekutu yang berada di sekitar ruin tersebut.
 3. Menghitung jarak ruin terhadap posisi robot saat ini.
 4. Memilih ruin dengan jarak paling dekat sebagai target konstruksi.
- **Fungsi Kelayakan:** ruin belum memiliki menara dan tidak sedang dipadati oleh Soldier sekutu.
- **Fungsi Obyektif:** meminimalkan jarak menuju lokasi ruin sehingga proses pembangunan menara dapat dilakukan secepat mungkin.
- **Fungsi Solusi:** ruin dengan jarak minimum dipilih sebagai target pembangunan menara dan robot akan bergerak menuju lokasi tersebut.

3.1.2. Greedy Pemilihan Target Pewarnaan Tile

Tujuan strategi ini adalah robot melakukan ekspansi wilayah melalui pewarnaan tile. Strategi greedy digunakan untuk menentukan tile yang memberikan dampak ekspansi wilayah terbesar dalam satu aksi pewarnaan.

- **Himpunan Kandidat:** seluruh tile dalam radius serangan robot yang dapat diwarnai.
- **Himpunan Solusi:** satu tile target yang dipilih untuk diwarnai pada giliran berjalan.
- **Fungsi Seleksi:**
 1. Memberikan bobot dasar lebih tinggi pada tile EMPTY, lalu ENEMY.
 2. Menambahkan skor jika tile berada di sekitar ruin yang sedang menjadi target pembangunan menara.
 3. Menambahkan skor apabila warna tile tidak sesuai dengan pola mark pembangunan menara.
 4. Menghindari tile yang sudah berwarna milik sekutu untuk mengurangi pemborosan paint.
- **Fungsi Kelayakan:** robot harus siap melakukan aksi (`isActionReady`) dan tile berada dalam jangkauan serangan.
- **Fungsi Obyektif:** memaksimalkan pertambahan wilayah sekutu pada setiap aksi pewarnaan.
- **Fungsi Solusi:** tile dengan skor heuristik tertinggi dipilih untuk diwarnai oleh robot.

3.1.3. Greedy Pemilihan Arah Gerak Eksplorasi

Tujuan strategi ini adalah untuk melakukan aksi pewarnaan, robot menentukan arah gerak yang paling menguntungkan untuk giliran berikutnya. Pendekatan ini menggunakan evaluasi *look-ahead* satu langkah, yaitu robot memperkirakan potensi pewarnaan tile dari posisi baru setelah bergerak.

- **Himpunan Kandidat:** seluruh arah gerakan yang valid (`canMove`) dari delapan arah utama.
- **Himpunan Solusi:** satu arah gerak terbaik yang dipilih untuk reposisi robot.
- **Fungsi Seleksi:**
 1. Menghitung potensi tile kosong yang dapat diwarnai dari posisi baru.
 2. Menambahkan skor jika terdapat tile musuh di sekitar posisi baru.
 3. Menambahkan skor jika langkah mendekati target eksplorasi (`exploreTarget`).
 4. Menambahkan skor jika langkah mendekati ruin yang sedang menjadi target pembangunan.
 5. Memberikan penalti pada tile yang sudah berwarna milik sekutu agar tidak terjadi overpaint.

- **Fungsi Kelayakan:** robot harus siap bergerak (`isMovementReady`) dan arah gerak valid.
- **Fungsi Obyektif:** memaksimalkan potensi ekspansi wilayah pada langkah berikutnya sambil menjaga penyebaran unit.
- **Fungsi Solusi:** robot bergerak ke arah dengan skor tertinggi untuk membuka peluang pewarnaan tile pada giliran selanjutnya.

3.1.4. Greedy Penentuan Pusat Serangan Area (*Splasher*)

Tujuan strategi ini adalah untuk menentukan titik pusat serangan terbaik berdasarkan kondisi tile di sekitar titik tersebut. Setiap kandidat lokasi serangan dievaluasi dengan menghitung potensi jumlah tile yang dapat diwarnai atau direbut dari musuh dalam radius efek splash.

- **Himpunan Kandidat:** seluruh tile di sekitar robot yang berada dalam jangkauan serangan area *Splasher*.
- **Himpunan Solusi:** satu titik pusat serangan yang memberikan dampak pewarnaan area terbesar.
- **Fungsi Seleksi:**
 1. Menghitung jumlah tile EMPTY di sekitar kandidat pusat serangan.
 2. Menambahkan skor lebih tinggi jika terdapat tile ENEMY yang berada dekat pusat serangan.
 3. Menambahkan skor tambahan apabila beberapa tile musuh berada dalam radius efek maksimum.
 4. Memberikan penalti kecil jika area tersebut sudah didominasi oleh tile sekutu.
- **Fungsi Kelayakan:** robot harus siap melakukan aksi (`isActionReady`) dan target berada dalam jangkauan serangan.
- **Fungsi Obyektif:** memaksimalkan jumlah tile yang dapat diwarnai atau direbut dari musuh dalam satu aksi serangan.
- **Fungsi Solusi:** titik pusat serangan dengan skor heuristik tertinggi dipilih sebagai target serangan area.

3.1.5. Greedy Pembersihan Area Musuh (*Mopper*)

Tujuan strategi ini adalah untuk menentukan target pembersihan tile musuh yang memberikan manfaat terbesar bagi tim. Robot mengevaluasi tile musuh di sekitarnya dan memilih target dengan nilai heuristik tertinggi.

- **Himpunan Kandidat:** seluruh tile yang memiliki warna milik musuh dan berada dalam jangkauan serangan robot.
- **Himpunan Solusi:** satu tile musuh yang dipilih sebagai target pembersihan.
- **Fungsi Seleksi:**

1. Memberikan skor lebih tinggi pada tile musuh yang berada dekat dengan posisi robot.
 2. Menambahkan skor jika tile musuh berada di sekitar ruin, karena area tersebut memiliki nilai strategis untuk pembangunan menara.
 3. Menambahkan skor jika pembersihan tile membuka peluang ekspansi wilayah pada langkah berikutnya.
- **Fungsi Kelayakan:** robot harus siap melakukan aksi dan target berada dalam jangkauan serangan.
 - **Fungsi Obyektif:** memaksimalkan stabilitas wilayah sekutu dengan menghilangkan pengaruh cat musuh pada area penting.
 - **Fungsi Solusi:** tile musuh dengan skor heuristik tertinggi dipilih sebagai target pembersihan.

3.1.6. Greedy Pemilihan Arah Gerak Unit Pendukung

Tujuan strategi ini adalah untuk memilih arah gerak yang memberikan dukungan terbaik terhadap Soldier maupun Splasher yang sedang melakukan ekspansi wilayah.

- **Himpunan Kandidat:** seluruh arah gerakan yang valid dari posisi robot.
- **Himpunan Solusi:** satu arah gerakan terbaik yang dipilih untuk reposisi unit pendukung.
- **Fungsi Seleksi:**
 1. Memberikan prioritas gerak menuju musuh yang berada di sekitar robot.
 2. Memberikan skor tambahan jika arah gerak mendekati Soldier sekutu yang sedang melakukan ekspansi.
 3. Menambahkan skor jika area di sekitar posisi baru didominasi oleh tile sekutu.
 4. Memberikan penalti jika arah gerak menuju wilayah yang didominasi oleh cat musuh.
- **Fungsi Kelayakan:** robot harus siap bergerak (`isMovementReady`) dan arah gerak valid.
- **Fungsi Obyektif:** menjaga kestabilan wilayah sekutu sekaligus mendukung ekspansi unit utama.
- **Fungsi Solusi:** robot bergerak menuju arah dengan skor heuristik tertinggi.

3.1.7. Mekanisme Anti-Stuck dan Stabilitas Pergerakan

Tujuan strategi ini adalah untuk mengatasi permasalahan ini, bot utama menerapkan mekanisme deteksi pergerakan berulang dengan menyimpan riwayat posisi robot dalam beberapa langkah terakhir. Dalam

kondisi ini, sistem akan mengubah target eksplorasi robot agar robot dapat keluar dari area tersebut dan melanjutkan proses ekspansi wilayah.

- **Himpunan Kandidat:** seluruh arah eksplorasi yang memungkinkan robot keluar dari area stuck.
- **Himpunan Solusi:** satu arah eksplorasi alternatif yang berbeda dari pola pergerakan sebelumnya.
- **Fungsi Seleksi:**
 1. Mengidentifikasi pola pergerakan berulang dalam beberapa langkah terakhir.
 2. Mengganti target eksplorasi apabila robot tidak membuat kemajuan pergerakan.
 3. Memilih arah eksplorasi alternatif yang menjauhi area stuck.
- **Fungsi Kelayakan:** robot berada dalam kondisi stuck dan masih memiliki opsi pergerakan.
- **Fungsi Obyektif:** menjaga kelancaran eksplorasi wilayah dan menghindari pemborosan langkah robot.
- **Fungsi Solusi:** robot mengubah target eksplorasi dan bergerak menuju arah baru yang membuka jalur ekspansi wilayah.

3.2. Bot Alternatif 1

Bot Alternatif 1 dirancang dengan fokus utama untuk memaksimalkan penguasaan wilayah melalui pewarnaan *tile* secepat mungkin. Berbeda dengan bot yang berorientasi *resource pattern* atau pembangunan menara, bot ini menempatkan aksi pewarnaan sebagai prioritas utama setiap giliran. Pendekatan *greedy* diterapkan pada dua keputusan inti: pemilihan target *tile* untuk diwarnai dan pemilihan arah gerak untuk ekspansi area berikutnya.

3.2.1. Greedy Pemilihan Target Pewarnaan Tile

Tujuan strategi ini adalah memilih *tile* yang memberi dampak ekspansi terbesar dalam satu aksi. Soldier mengevaluasi *tile* yang belum berwarna, dalam jangkauan serang, lalu memilih *tile* dengan skor heuristik tertinggi. Elemen algoritma *greedy* adalah:

- **Himpunan Kandidat:** seluruh *tile* dalam radius serang dari posisi robot saat ini yang dapat diwarnai (*canAttack*) dan belum berwarna.
- **Himpunan Solusi:** satu *tile* target terbaik yang akan diwarnai pada giliran berjalan.
- **Fungsi Seleksi:**
 5. Memberi bobot dasar lebih tinggi pada *tile* EMPTY, lalu ENEMY.

6. Menambahkan *frontier expansion* (jumlah tetangga berwarna di sekitar *tile* target), karena *tile* seperti ini membuka peluang pewarnaan lanjutan yang lebih besar.
 7. Menambahkan progres ke arah area eksplorasi (*exploreTarget*) agar ekspansi tetap terarah.
 8. Menambahkan jarak dari tower asal (*homeTower*) untuk mendorong perluasan keluar basis.
- **Fungsi Kelayakan:** robot harus siap aksi (*isActionReady*), target valid dalam jangkauan serang, dan paint tidak berada pada kondisi kritis.
 - **Fungsi Obyektif:** memaksimalkan pertambahan cakupan wilayah per aksi paint dan mempercepat dominasi peta.
 - **Fungsi Solusi:** *tile* dengan skor tertinggi diwarnai, jika ada mark sekunder, bot menyesuaikan warna (*ALLY_SECONDARY*) agar konsisten dengan mark yang ada.

3.2.2. Greedy Pemilihan Arah Gerak Ekspansi

Setelah aksi paint, bot menentukan langkah gerak yang paling menguntungkan untuk giliran berikutnya. Pendekatan ini bersifat *look-ahead* satu langkah: bot menilai posisi setelah bergerak lalu memperkirakan kualitas target paint yang bisa diambil dari posisi baru tersebut. Elemen greedy nya

- **Himpunan Kandidat:** seluruh arah gerak yang valid (*canMove*) dari 8 arah utama.
- **Himpunan Solusi:** satu arah gerak terbaik yang dipilih untuk reposisi.
- **Fungsi Seleksi:**
 1. Menghitung projected attack score dari posisi baru (seberapa bagus target paint potensial setelah bergerak).
 2. Menambah skor jika di sekitar posisi baru terdapat banyak *tile* belum berwarna (peluang ekspansi lanjutan).
 3. Menambah skor jika langkah mendekati *exploreTarget*; memberi penalti jika menjauh.
 4. Memberi penalti pada *tile* tujuan yang sudah berwarna milik sendiri (menghindari overpaint area aman).
 5. Memberi penalti kepadatan di sekitar tujuan (mengurangi penumpukan robot).
- **Fungsi Kelayakan:** robot harus siap bergerak (*isMovementReady*) dan arah tersebut valid.
- **Fungsi Obyektif:** memaksimalkan potensi pewarnaan giliran selanjutnya sambil menjaga sebaran unit.

- **Fungsi Solusi:** robot bergerak ke arah dengan skor tertinggi, lalu jika masih memungkinkan, mewarnai tile posisi saat ini sebagai aksi penutup giliran.

3.3. Bot Alternatif 2

Bot Alternatif 2 (Resource Specialist) dirancang dengan fokus utama untuk memaksimalkan keuntungan ekonomi dan *resource* tim. Algoritma *greedy* diterapkan pada dua aksi strategis, yaitu saat pembentukan *Special Resource Pattern* (SRP) dan proses *upgrade* menara (tower).

3.3.1. Greedy Pencarian dan Pembentukan *Special Resource Pattern* (SRP)

Tujuan dari strategi ini adalah untuk mendapatkan suplai *chips* (uang) secepat mungkin dengan menduduki dan mengaktifkan *ruins* menggunakan pola warna khusus (SRP). Karena pencarian jalan (*pathfinding*) yang jauh akan menghabiskan banyak waktu (putaran) dan cat, pendekatan heuristik *greedy* digunakan untuk mempercepat keputusan bot. Bot (tipe Soldier) akan mencari *ruins* terdekat dari posisinya untuk dikonversi menjadi *resource pattern center*. Berikut adalah penjabaran elemen algoritma *greedy*-nya:

- **Himpunan Kandidat:** Seluruh lokasi *ruins* yang terdeteksi oleh jangkauan sensor robot.
- **Himpunan Solusi:** *Ruins* yang belum dijadikan *resource pattern center* dan sedang dituju atau diproses oleh bot.
- **Fungsi Seleksi:** Mencari target secara heuristik dengan memilih *ruins* yang belum ditempati menara atau berstatus SRP, lalu membandingkan jaraknya menggunakan fungsi *distanceSquaredTo*. *Ruins* dengan nilai jarak terendah (terdekat) akan diprioritaskan.
- **Fungsi Kelayakan:** Memastikan ronde permainan saat ini masih berada di bawah ronde 1850 (agar bonus SRP sempat aktif sebelum permainan berakhir) dan bot memiliki kemampuan untuk memanggil fungsi *canMarkResourcePattern()*.
- **Fungsi Obyektif:** Meminimalkan jarak dan waktu tempuh menuju *ruins* untuk mempercepat produksi *chips*.
- **Fungsi Solusi:** Seluruh area 5x5 di sekitar *ruins* telah diwarnai sesuai pola dan *canCompleteResourcePattern()* dapat dieksekusi setelah mengumpulkan 200 *chips*.

3.3.2. Greedy Pemilihan Prioritas *Upgrade Tower*

Apabila di area sekitar bot tidak ditemukan *ruins* yang valid atau bot sudah melewati ronde batas aman SRP, bot akan beralih ke tugas sekunder (tugas *fallback*) yakni meningkatkan level menara tim. Algoritma *greedy* dalam penentuan menara yang akan di-*upgrade* mempertimbangkan faktor keamanan secara absolut. Elemen algoritma *greedy* pada fungsi ini meliputi:

- **Himpunan Kandidat:** Seluruh unit bertipe menara yang tergabung dalam tim sekutu dan berada dalam batas jangkauan penglihatan (*vision*).
- **Himpunan Solusi:** Satu unit menara sekutu terpilih yang siap ditingkatkan levelnya.
- **Fungsi Seleksi:** Memilih menara kandidat yang memiliki nilai nyawa (*Health Points/HP*) paling tinggi. Asumsi *greedy*-nya adalah menara dengan HP penuh memiliki peluang hidup lebih panjang di ronde berikutnya, sehingga investasi *upgrade* tidak akan sia-sia jika dibandingkan dengan *upgrade* menara yang hampir hancur diserang musuh. Jika terdapat dua menara dengan HP yang identik, fungsi akan menggunakan *tie-breaker* dengan memilih jarak fisik yang paling dekat dengan bot.
- **Fungsi Kelayakan:** Syarat mutlaknya adalah unit kandidat memang berjenis menara (*isTowerType()*), memiliki kapasitas untuk naik level (*canUpgradeType()*), dan fungsi aksi *canUpgradeTower()* me-return *true*.
- **Fungsi Obyektif:** Memaksimalkan utilitas (*utility*) menara sekutu terkuat untuk meningkatkan dominasi pada peta.
- **Fungsi Solusi:** *Upgrade* berhasil dilakukan ke target terpilih dan indikator "Tower Upgraded" tercatat pada garis waktu (*timeline*).

4. Implementasi dan Pengujian

Pada bagian ini, kami akan mendeskripsikan implementasi spesifik dari rancangan *greedy* yang dibahas pada Bab 3 ke dalam kode Java pada platform permainan *Battlecode*.

Implementasi

4.1. Bot Utama

4.1.1. Manajemen Unit

Pada implementasi utama, kelas **RobotPlayer** main-bot bertanggung jawab mengatur seluruh perilaku unit robot melalui mekanisme switch-case berdasarkan tipe unit robot. setiap jenis robot memiliki spesialisasi tugas yang berbeda untuk memaksimalkan efisiensi ekspansi wilayah dan penguasaan peta. Robot Soldier bertindak sebagai unit pekerja utama yang melakukan ekspansi wilayah dengan mewarnai petak kosong serta mengeksekusi konstruksi menara dari objek ruins, Robot Splasher difokuskan pada penguasaan area secara luas dengan mekanisme serangan area, Robot Mopper berfungsi sebagai pendukung yang membersihkan cat musuh pada area sekitar dan membantu menjaga stabilitas warna wilayah sekutu, serta Tower berperan sebagai pusat produksi robot dan pertahanan wilayah.

4.1.2. Greedy Ruin Claiming

Pendekatan utama dalam membangun menara adalah menggunakan strategi greedy untuk memilih ruin terdekat yang masih tersedia. Implementasi ini dilakukan dalam fungsi *findBestRuin(RobotController rc)* yang melakukan pemindaian terhadap seluruh objek ruins di sekitar robot. Setiap kandidat ruin akan melalui proses seleksi berdasarkan beberapa kriteria kelayakan, yaitu Ruin belum memiliki menara, Jumlah robot sekutu yang sedang mengklaim ruin tidak terlalu banyak, serta ruin memiliki jarak minimum dari posisi robot saat ini. Fungsi seleksi greedy ini mencari ruin dengan jarak terkecil dari robot sehingga proses pembangunan menara dapat dilakukan secepat mungkin.

Setelah proses pemilihan kandidat dilakukan, algoritma greedy akan memilih *ruin* dengan jarak terpendek dari posisi robot saat ini. Pendekatan ini memungkinkan robot untuk membangun menara dengan waktu yang lebih efisien karena meminimalkan jarak perjalanan menuju lokasi konstruksi. Ruin yang terpilih kemudian akan disimpan sebagai target konstruksi dan diproses pada tahap berikutnya yaitu pengisian pola menara.

```
static MapInfo findBestRuin(RobotController rc, MapInfo[] nearbyTiles) throws
GameActionException {
    MapInfo best = null;
```

```
int bestDist = Integer.MAX_VALUE;

for (MapInfo tile : nearbyTiles) {
    if (!tile.hasRuin()) continue;

    RobotInfo[] botsOnRuin = rc.senseNearbyRobots(tile.getMapLocation(), 2,
null);
    boolean hasTower = false;
    int allySoldiers = 0;

    for (RobotInfo r : botsOnRuin) {
        if (r.getType().isTowerType()) {
            hasTower = true;
            break;
        }
        if (r.getTeam() == rc.getTeam() && r.getType() == UnitType.SOLDIER){
            allySoldiers++;
        }
    }

    if (!hasTower && allySoldiers < 2) {
        int d = rc.getLocation().distanceSquaredTo(tile.getMapLocation());
        if (d < bestDist) {
            bestDist = d;
            best = tile;
        }
    }
}

return best;
}
```

4.1.3. Greedy Tower Construction

Pada logika *greedy* ini, setelah ruin berhasil ditemukan, robot akan menjalankan proses pembangunan menara melalui fungsi *handleRuinClaiming()*. Proses ini dilakukan melalui beberapa tahapan yang dilakukan secara berulang hingga pola menara terpenuhi sepenuhnya. Tahap pertama adalah menandai pola menara menggunakan fungsi *markTowerPattern()*. Setelah pola ditandai, robot akan mulai mengisi setiap petak dalam pola tersebut dengan warna yang sesuai dengan jenis pola yang ditentukan oleh sistem. Pemilihan jenis tower ini menggunakan pendekatan heuristik berdasarkan kondisi ekonomi tim. Jika jumlah chips yang dimiliki tim banyak, maka robot akan memilih membangun *Paint Tower* untuk mempercepat ekspansi wilayah. Sebaliknya, apabila jumlah *chips* masih terbatas maka robot akan membangun *Money Tower* guna meningkatkan produksi sumber daya ekonomi tim.

```
static UnitType chooseTowerType(RobotController rc) {
    return rc.getChips() > 2000
        ? UnitType.LEVEL_ONE_PAINT_TOWER
        : UnitType.LEVEL_ONE_MONEY_TOWER;
}
```

4.1.4. Greedy Area Expansion

Pada logika greedy ini berfungsi untuk memperluas wilayah permainan secara efektif, *Robot Soldier* menggunakan logika strategi *greedy expansion* dalam menentukan arah pergerakan. Implementasi strategi ini dilakukan melalui fungsi *findBestExpansionDirection()* yang mengevaluasi setiap kemungkinan arah gerakan robot. Setiap arah pergerakan akan diberikan skor berdasarkan jumlah petak kosong yang dapat diwarnai, jumlah petak musuh yang dapat direbut kembali, serta jarak terhadap target eksplorasi yang telah ditentukan sebelumnya. Dengan pendekatan logika ini, robot akan memilih arah pergerakan yang memberikan potensi ekspansi wilayah terbesar dalam setiap langkah.

```
for (Direction dir : directions) {
    if (!rc.canMove(dir)) continue;

    MapLocation nextLoc = myLoc.add(dir);
    int score = 0;

    for (MapInfo tile : nearbyTiles) {
        if (nextLoc.distanceSquaredTo(tile.getMapLocation()) <= ATTACK_RANGE_SQ)
        {
            PaintType p = tile.getPaint();

            if (p == PaintType.EMPTY) score += 4;
            else if (p.isEnemy()) score += 2;
        }
    }

    if (score > bestScore) {
        bestScore = score;
        bestDir = dir;
    }
}
```

4.1.5. Greedy Splash Attack

Pada logika *greedy* ini, *Robot Splasher* memiliki peran penting dalam mempercepat dalam mempercepat dominasi wilayah melalui serangan area. Strategi ini untuk menentukan lokasi serangan terbaik, bot menggunakan pendekatan heuristik yang mengevaluasi setiap kandidat titik serangan di sekitar robot. Setiap kandidat akan dihitung skornya berdasarkan jumlah petak kosong maupun petak musuh yang berada dalam radius efek splash. Petak kosong akan memberikan keuntungan karena dapat diwarnai oleh tim, sedangkan petak musuh meningkatkan skor karena dapat direbut kembali. Sebaliknya, petak sekutu akan mengurangi nilai skor agar robot tidak membuang sumber daya *paint* pada wilayah yang sudah dikuasai.

```
for (MapInfo tile : nearbyTiles) {
    int d = center.distanceSquaredTo(tile.getMapLocation());
    if (d > 4) continue;
```

```
PaintType p = tile.getPaint();

if (p == PaintType.EMPTY) score += 5;
else if (p.isEnemy()) score += 6;
else if (p.isAlly()) score -= 1;
}
```

4.1.6. Manajemen Cat

Mekanisme pengisian ulang ini dilakukan menggunakan fungsi **transferPaint()** yang memungkinkan robot mengambil paint dari menara sekutu. Dengan adanya mekanisme ini, robot dapat menghindari kondisi kehabisan paint yang dapat menyebabkan unit tidak dapat menyerang ataupun mewarnai wilayah. Selain itu, unit Mopper juga memiliki kemampuan untuk mendistribusikan paint kepada robot sekutu yang berada di sekitarnya sehingga menjaga kestabilan sumber daya dalam tim.

```
if (rc.getPaint() < SOLDIER_REFILL_THRESHOLD) {
    if (!tryWithdrawFromNearbyTower(rc))
        moveTowards(rc, findNearestTower(rc));
    return;
}
```

4.2. Bot Alternatif 1

4.2.1. Manajemen Unit

Pada implementasi utama, kelas **RobotPlayer** bot alternatif 1, mengatur perilaku tiap unit menggunakan struktur switch-case berdasarkan tipe robot. Unit Soldier menjadi aktor utama ekspansi wilayah dengan memprioritaskan pemilihan target pewarnaan dan arah gerak terbaik melalui fungsi penilaian (scoring). Unit Tower bertugas memproduksi pasukan dengan komposisi dominan Soldier (sekitar 6 dari 8 spawn), diselingi Mopper dan Splasher sebagai unit pendukung. Unit Mopper digunakan untuk membersihkan cat musuh di sekitar area konflik, sedangkan Splasher membantu pewarnaan area secara menyebar. Selain itu, semua unit menerapkan manajemen paint adaptif: ketika paint berada di bawah ambang batas, unit akan kembali ke menara terdekat untuk refill sebelum melanjutkan ekspansi.

4.2.2. Greedy Pemilihan Target Pewarnaan

Greedy utama pertama berada pada fungsi **chooseBestPaintTarget()**. Bot mengevaluasi seluruh tile yang belum berwarna dalam jangkauan serang, lalu memilih kandidat dengan skor tertinggi.

```
static MapLocation chooseBestPaintTarget(RobotController rc, MapLocation from,
MapInfo[] nearbyTiles) {
    MapLocation best = null;
```

```
int bestScore = Integer.MIN_VALUE;
boolean fromCurrentLocation = from.equals(rc.getLocation());

for (MapInfo tile : nearbyTiles) {
    MapLocation loc = tile.getMapLocation();
    if (from.distanceSquaredTo(loc) > 9) {

        continue;
    }
    if (fromCurrentLocation && !rc.canAttack(loc)) {
        continue;
    }

    PaintType paint = tile.getPaint();
    if (paint.isAlly()) {

        continue;
    }

    int score = 0;
    if (paint == PaintType.EMPTY) {
        score += 30;
    } else if (paint.isEnemy()) {
        score += 22;
    } else {
        score += 8;
    }

    int frontier = countNonAllyNeighbors(loc, nearbyTiles);
    score += frontier * 4;

    if (homeTower != null) {
        score += Math.min(12, loc.distanceSquaredTo(homeTower) / 12);
    }

    if (exploreTarget != null) {
        int cur = from.distanceSquaredTo(exploreTarget);
        int nxt = loc.distanceSquaredTo(exploreTarget);
        if (nxt < cur) {
            score += 4;
        }
    }

    if (score > bestScore) {
        bestScore = score;
        best = loc;
    }
}
return best;
}
```

Bot terlebih dahulu membentuk himpunan kandidat dari seluruh tile di sekitar yang masih berada dalam jangkauan serang, valid untuk diserang, dan belum berwarna. Setiap kandidat kemudian diberi nilai heuristik: tile kosong (EMPTY) memperoleh prioritas tertinggi karena paling efektif menambah area kontrol, tile musuh (ENEMY) juga bernilai tinggi karena sekaligus merebut wilayah lawan, lalu skor ditambah dengan komponen frontier

expansion yang dihitung dari banyaknya tetangga non-ally di sekitar tile target.

4.2.3. Greedy Pemilihan Arah Gerak

Greedy utama kedua ada di fungsi `chooseBestMoveDirection()`. Setiap arah gerak valid diberi skor, lalu dipilih arah dengan nilai terbesar.

```
static Direction chooseBestMoveDirection(RobotController rc, MapInfo[]
nearbyTiles) throws GameActionException {
    MapLocation myLoc = rc.getLocation();
    Direction bestDir = null;
    int bestScore = Integer.MIN_VALUE;

    for (Direction d : DIRECTIONS) {
        if (!rc.canMove(d)) {
            continue;
        }

        MapLocation next = myLoc.add(d);
        int score = 0;

        MapLocation projectedAttack = chooseBestPaintTarget(rc, next,
nearbyTiles);
        if (projectedAttack != null) {
            score += scoreProjectedAttack(next, projectedAttack,
nearbyTiles);
        } else {
            score -= 3;
        }

        score += countNonAllyInRadius(next, nearbyTiles, 8) * 2;

        if (exploreTarget != null) {
            int curDist = myLoc.distanceSquaredTo(exploreTarget);
            int nextDist = next.distanceSquaredTo(exploreTarget);
            if (nextDist < curDist) {
                score += 10;
            } else if (nextDist > curDist) {
                score -= 2;
            }
        }

        if (lastMoveDir != null && d == lastMoveDir) {
            score += 2;
        }

        if (rc.senseMapInfo(next).getPaint().isAlly()) {
            score -= 2;
        }

        int crowd = countNearbyAllies(rc, next);
        score -= crowd * 2;

        if (score > bestScore) {
            bestScore = score;
            bestDir = d;
        }
    }
}
```



```
if (bestDir == null) {
    int start = rng.nextInt(DIRECTIONS.length);
    for (int i = 0; i < DIRECTIONS.length; i++) {
        Direction d = DIRECTIONS[(start + i) % DIRECTIONS.length];
        if (rc.canMove(d)) {
            return d;
        }
    }
}
return bestDir;
}
```

Untuk memilih arah gerak terbaik melalui fungsi *chooseBestMoveDirection()*. Bot mengevaluasi seluruh arah yang valid (*canMove*) lalu memberi skor pada setiap kandidat posisi berikutnya (*next*). Inti penilaiannya adalah *look-ahead* satu langkah: dari posisi baru itu, bot memperkirakan seberapa bagus peluang paint selanjutnya dengan memanggil *chooseBestPaintTarget()* dan menambahkan *scoreProjectedAttack()*.

4.2.4. Greedy Pendukung Unit Splasher

Pada fungsi *chooseBestSplashTarget(...)*, Splasher memilih pusat serangan area berdasarkan skor lokal.

```
static MapLocation chooseBestSplashTarget(RobotController rc, MapInfo[]
nearbyTiles) {
    MapLocation best = null;
    int bestScore = Integer.MIN_VALUE;
    MapLocation myLoc = rc.getLocation();

    for (MapInfo cand : nearbyTiles) {
        MapLocation center = cand.getMapLocation();
        if (myLoc.distanceSquaredTo(center) > 4 || !rc.canAttack(center)) {
            continue;
        }
        int score = 0;
        for (MapInfo tile : nearbyTiles) {
            if (center.distanceSquaredTo(tile.getMapLocation()) > 4) {
                continue;
            }
            PaintType p = tile.getPaint();
            if (p == PaintType.EMPTY) {
                score += 3;
            } else if (p.isEnemy()) {
                score += 5;
            } else {
                score -= 1;
            }
        }
        if (score > bestScore) {
            bestScore = score;
            best = center;
        }
    }
    return best;
}
```

```
}
```

Unit Splasher pada Bot Alternatif 1 memakai pendekatan greedy untuk menentukan pusat serangan area terbaik melalui fungsi *chooseBestSplashTarget()*. Bot membentuk kandidat dari tile yang masih berada dalam jangkauan serang Splasher, lalu untuk setiap kandidat menghitung nilai area di radius splash-nya. Nilai akan naik jika area tersebut mencakup banyak tile EMPTY (potensi ekspansi) dan terutama tile ENEMY (merebut kontrol lawan), sedangkan tile yang sudah berwarna milik sendiri diberi penalti agar serangan tidak boros pada wilayah sendiri.

4.3. Bot Alternatif 2

4.3.1. Manajemen Unit

Pada implementasi utama, kelas **RobotPlayer** Alternatif 2 mengatur logika untuk tiap unit menggunakan konstruksi switch-case. Karena fokus utama bot ini adalah spesialisasi sumber daya, unit Soldier ditugaskan sebagai pekerja keras (worker) untuk mewarnai peta dan mengeksekusi SRP. Sementara itu, unit bertipe menara diberikan instruksi dasar untuk memproduksi (spawn) robot sekutu dengan proporsi tertentu (1 Mopper setiap 3 Soldier) guna menjaga kebersihan di wilayah SRP agar pola warna tidak rusak oleh cat musuh.

4.3.2. Greedy Special Resource Pattern (SRP)

Implementasi logika pencarian terdekat ini ditempatkan pada fungsi *kerjakanSRP(RobotController rc)*. Bot mendeteksi *ruins* dengan *senseNearbyRuins(-1)* dan melakukan perulangan pada senarai *ruins* tersebut. Fungsi penyeleksian di bawah ini mengukur jarak tiap kandidat, lalu melempar kandidat yang tidak layak (sudah menjadi pusat SRP atau diduduki menara), hingga menemukan titik terdekat:

>. *bahasa Java*

```
// Greedy: cari ruins terdekat yang BELUM jadi resource pattern center
MapLocation targetRuin = null;
int jarakTerdekat = Integer.MAX_VALUE;
for(int i = 0; i < semuaRuins.length; i++){
    MapLocation lokasiRuin = semuaRuins[i];
    MapInfo infoRuin = rc.senseMapInfo(lokasiRuin);

    // Fungsi Kelayakan: Lompati jika sudah jadi SRP Center
    if(infoRuin.isResourcePatternCenter()){
        continue;
    }
    // Fungsi Kelayakan: Lompati jika sudah ada bangunan (Tower)
    if(rc.canSenseRobotAtLocation(lokasiRuin)){
        RobotInfo robotDisitu = rc.senseRobotAtLocation(lokasiRuin);
        if(robotDisitu != null && robotDisitu.getType().isTowerType()){
```

```
        continue;
    }
}
// Fungsi Seleksi: Minimasi jarak terpendek
int jarak = posisiSekarang.distanceSquaredTo(lokasiRuin);
if(jarak < jarakTerdekat){
    jarakTerdekat = jarak;
    targetRuin = lokasiRuin;
}
}
```

Apabila *ruins* berhasil dicapai, bot mengeksekusi fungsi *mark* dan secara metodis mengeksekusi pengulangan bersarang (*nested loop*) dalam ukuran matriks 5x5 menggunakan *translate(offsetX,offsetY)* untuk menggoreskan pola cat utama (primer) dan kedua (sekunder). Puncak dari tugas ini adalah verifikasi kepemilikan modal (*chips* ≥ 200) untuk menyelesaikan konstruksi dengan *completeResourcePattern()*.

4.3.3. Greedy Upgrade Tower

Untuk kasus di mana ruins sulit ditemukan atau ronde telah melampaui tenggat efektif SRP (putaran ke-1850), implementasi berlanjut pada fallback algoritma heuristik pada fungsi *cobaUpgradeTower(RobotController rc)*. Bot akan mengecek status seluruh unit (robot) yang berada di dekatnya.

>. bahasa Java

```
for(int i = 0; i < robotSekitar.length; i++){
    RobotInfo info = robotSekitar[i];
    UnitType tipeMenara = info.getType();

    // Fungsi Kelayakan
    if(!tipeMenara.isTowerType() || !tipeMenara.canUpgradeType()){
        continue;
    }

    int hpMenara = info.getHealth();
    int jarakMenara = posisiSekarang.distanceSquaredTo(info.getLocation());

    // Fungsi Seleksi: Maksimasi HP tertinggi, Tie-break: minimasi jarak
    if(hpMenara > hpTertinggi || (hpMenara == hpTertinggi && jarakMenara <
    jarakKeMenara)){
        hpTertinggi = hpMenara;
        jarakKeMenara = jarakMenara;
        targetMenara = info.getLocation();
    }
}
```

Seperti yang terlihat pada kode, fungsi seleksi memprioritaskan unit menara yang kondisinya paling utuh (*hpMenara* $>$ *hpTertinggi*). Keputusan logis heuristik (*tie-break*) juga diaplikasikan. Apabila kondisi HP seimbang antardua kandidat menara, kode secara implisit akan memilih jarak tempuh yang lebih pendek untuk menekan konsumsi pergerakan komputasi robot (*hpMenara* $==$ *hpTertinggi* && *jarakMenara* $<$ *jarakKeMenara*).

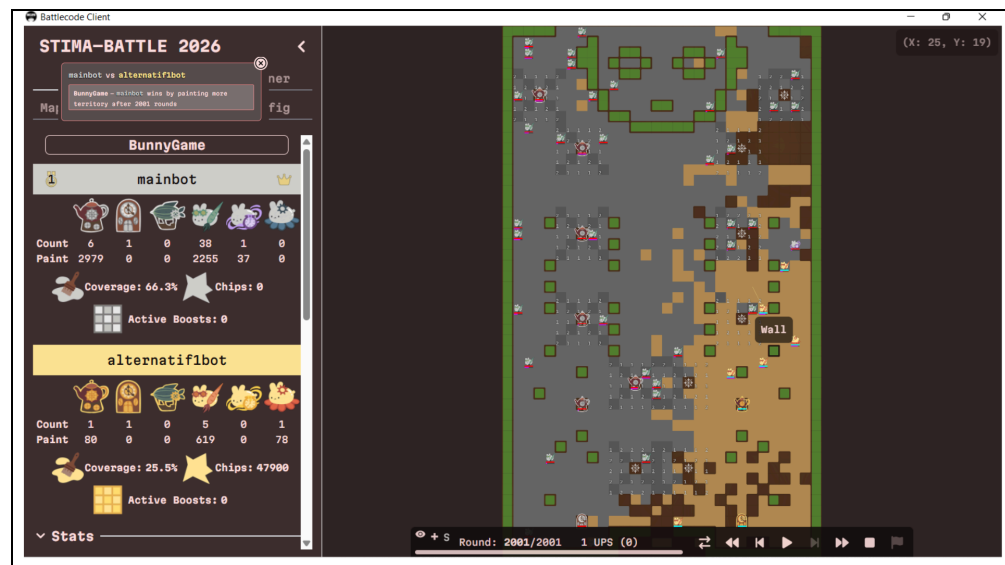
4.3.4. Manajemen Modul Cat (Paint Management)

Guna mendukung aksi pergerakan dan serangan *greedy* tersebut, manajemen logistik pada bot ini sangat kritis. Pada fungsi *runSoldier()*, proses pengawasan logistik menjadi langkah pertama. Apabila sisa kuantitas cat yang dimiliki bot telah merosot di bawah batas ambang 20% kapasitas utama ($catSaatIni < (catMaksimal / 5)$), unit terpaksa melepaskan aktivitas objektif terdekat untuk menjalankan fungsi darurat *cariMenaraUntukRefill(rc)*. Proses transfer memanfaatkan pertukaran argumen bernilai negatif *rc.transferPaint(menaraTerdekat, -catYangDiminta)* agar menara memompa suplai cat ke robot pekerja tersebut, mencegah kekosongan tenaga (*out of paint*) yang bisa melumpuhkan robot di tengah misi utamanya.

Pengujian

4.4. Hasil Uji Kasus 1

Spesifikasi Lingkungan dan Konfigurasi Lawan: Pengujian pada skenario pertama ini dikonfigurasi untuk mengevaluasi performa Bot Utama (*Mainbot*) ketika dihadapkan dengan Bot Alternatif 1 (*Alt1*). Arena yang ditetapkan sebagai lingkungan uji coba adalah peta "bunnygame".



Gambar 3. Hasil Uji Kasus Skenario 01.

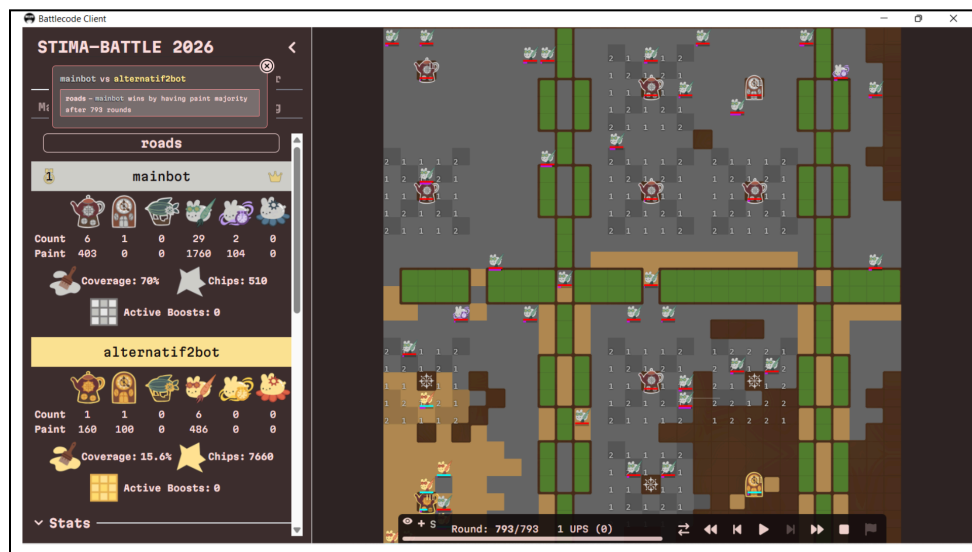
Berdasarkan tangkapan layar simulasi pada Gambar 3, dapat diamati hasil akhir dari Skenario Uji 1 yang mempertemukan mainbot (Tim A / Abu-abu) melawan agen penyerang alternatif1bot (Tim B / Cokelat). Simulasi ini dijalankan pada lingkungan peta "BunnyGame".

Pertandingan berjalan sengit dan mencapai batas maksimum putaran permainan (*turn limit*), yakni hingga ronde ke-2001 tanpa ada pihak yang berhasil mengeliminasi seluruh menara lawan. Oleh karena itu, sistem menentukan pemenang melalui mekanisme *tiebreaker* (penentuan seri), di mana tim dengan persentase penguasaan wilayah (jumlah ubin berwarna) terbanyak keluar sebagai pemenang.

Teks informasi pada antarmuka (*client*) secara eksplisit menyatakan bahwa "mainbot wins by painting more territory after 2001 rounds". Secara visual, hal ini didukung oleh dominasi warna abu-abu yang menutupi sebagian besar area peta, terutama di sisi kiri dan tengah arena. Hasil ini membuktikan bahwa algoritma heuristik *Greedy* pada mainbot yang difokuskan sebagai *Territorial Expander* (Penakluk Wilayah) berjalan sangat efektif dalam mengepung lawan dan mendominasi luasan peta secara konsisten hingga akhir permainan.

4.5. Hasil Uji Kasus 2

Spesifikasi Lingkungan dan Konfigurasi Lawan: Pengujian pada skenario kedua ini dikonfigurasi untuk mengevaluasi performa Bot Utama (*Mainbot*) ketika dihadapkan dengan Bot Alternatif 2 (*Alt2*). Arena yang ditetapkan sebagai lingkungan uji coba adalah peta "roads".



Gambar 4. Hasil Uji Kasus Skenario 02.

Berdasarkan tangkapan layar simulasi pada Gambar 4, Skenario Uji 2 yang mempertemukan mainbot (Tim Abu-abu) dengan alternatif2bot (Tim Kuning) pada peta "roads" berakhir dengan kemenangan mutlak untuk mainbot. Berbeda dengan uji kasus sebelumnya yang mencapai batas akhir putaran, pertandingan ini berakhir jauh lebih cepat, yakni pada putaran (*round*) ke-793.

Pesan pada antarmuka menyatakan bahwa "mainbot wins by having paint majority after 793 rounds". Hal ini terjadi karena mainbot berhasil mencapai kondisi

kemenangan absolut dengan menguasai tepat 70% luasan wilayah peta. Keberhasilan ini membuktikan bahwa algoritma heuristik ekspansi wilayah (termasuk *Center Rush* dan *Anti-Bias Arah*) pada mainbot bekerja dengan sangat agresif dan terarah sejak awal permainan.

Lebih lanjut, panel statistik pada gambar memberikan bukti empiris yang sangat jelas mengenai *trade-off* (pertukaran prioritas) dari kedua algoritma yang digunakan. Mainbot berhasil memenangkan wilayah (70%) namun memiliki sisa sumber daya yang sangat rendah, yakni hanya 510 *chips*. Sebaliknya, alternatif2bot yang bertindak sebagai *Resource Specialist* kalah telak dalam penguasaan wilayah (15,6%), namun berhasil menimbun kekayaan hingga 7.660 *chips*. Hasil ini memvalidasi rumusan masalah pada laporan, di mana fokus yang kaku pada formasi ekonomi (SRP) akan mengorbankan luasan area, sementara fokus pada dominasi area akan menguras kas ekonomi tim.

4.6. Hasil Uji Kasus 3

Spesifikasi Lingkungan dan Konfigurasi Lawan: Pengujian pada skenario kedua ini dikonfigurasi untuk mengevaluasi performa Bot Alternatif 1 (*Alt2*) ketika dihadapkan dengan Bot Alternatif 2 (*Alt2*). Arena yang ditetapkan sebagai lingkungan uji coba adalah peta "starburst".



Gambar 5. Hasil Uji Kasus Skenario 03.

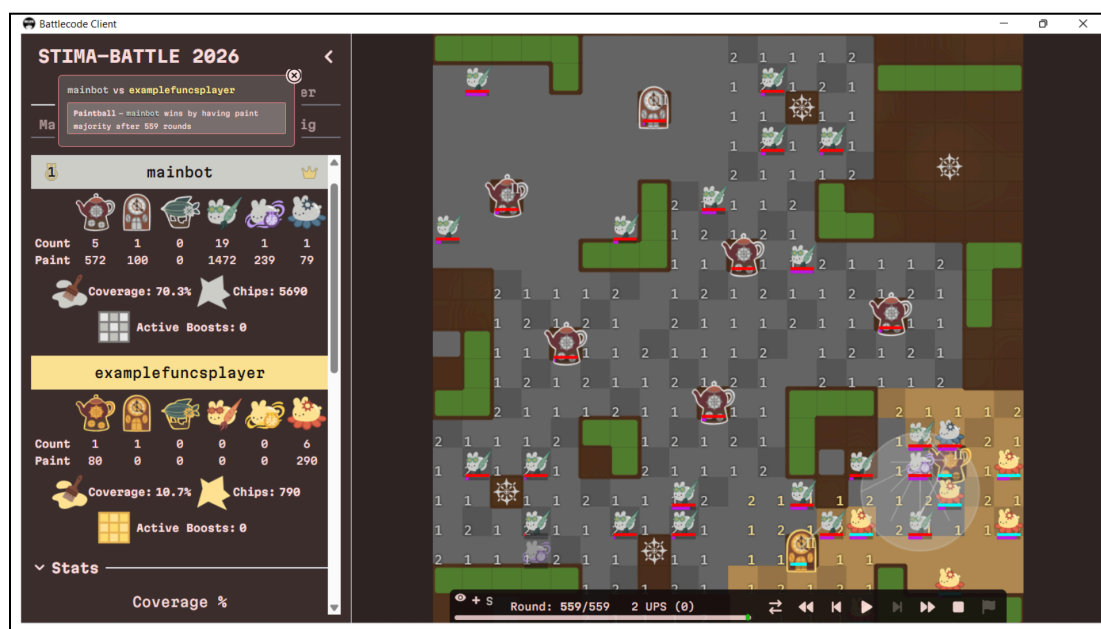
Berdasarkan tangkapan layar simulasi pada Gambar 5, Skenario Uji 3 berakhir dengan kemenangan absolut bagi agen alternatif1bot (Tim Abu-abu) atas alternatif2bot (Tim Kuning). Pertandingan ini tidak mencapai batas putaran maksimum (2000 ronde), melainkan diselesaikan secara dini pada putaran ke-1912. Pesan log sistem menunjukkan "alternatif1bot wins by having paint majority", yang

dipicu karena Tim Abu-abu berhasil menyentuh ambang batas kemenangan mutlak, yaitu 70% penguasaan luasan wilayah.

Analisis komparatif pada panel statistik mengungkap fenomena kompetisi yang menarik. Alternatif1bot tidak hanya mendominasi peta secara teritorial (70% berbanding 9%), tetapi juga unggul secara masif dalam akumulasi sumber daya ekonomi (50.630 *chips* berbanding 26.780 *chips*). Kekalahan ganda yang dialami oleh alternatif2bot yang sejatinya diarsiteki sebagai spesialis sumber daya (*Resource Specialist*) membuktikan sebuah kelemahan fundamental: strategi heuristik ekonomi (SRP) menjadi tidak efektif apabila agen kehilangan kontrol ruang gerak (*map control*) pada fase awal. Ekspansi agresif dari alternatif1bot sukses menyudutkan musuh ke area sempit, memutus akses alternatif2bot terhadap puing-puing (*ruins*) baru, dan pada akhirnya melumpuhkan potensi pertumbuhan ekonomi bot tersebut secara total.

4.7. Hasil Uji Kasus 4

Spesifikasi Lingkungan dan Konfigurasi Lawan: Pengujian pada skenario kedua ini dikonfigurasi untuk mengevaluasi performa Bot utama (*Mainbot*) ketika dihadapkan dengan Bot Contoh Asisten (*Examplefuncsplayer*). Arena yang ditetapkan sebagai lingkungan uji coba adalah peta "Paintball".



Gambar 6. Hasil Uji Kasus Skenario 04.

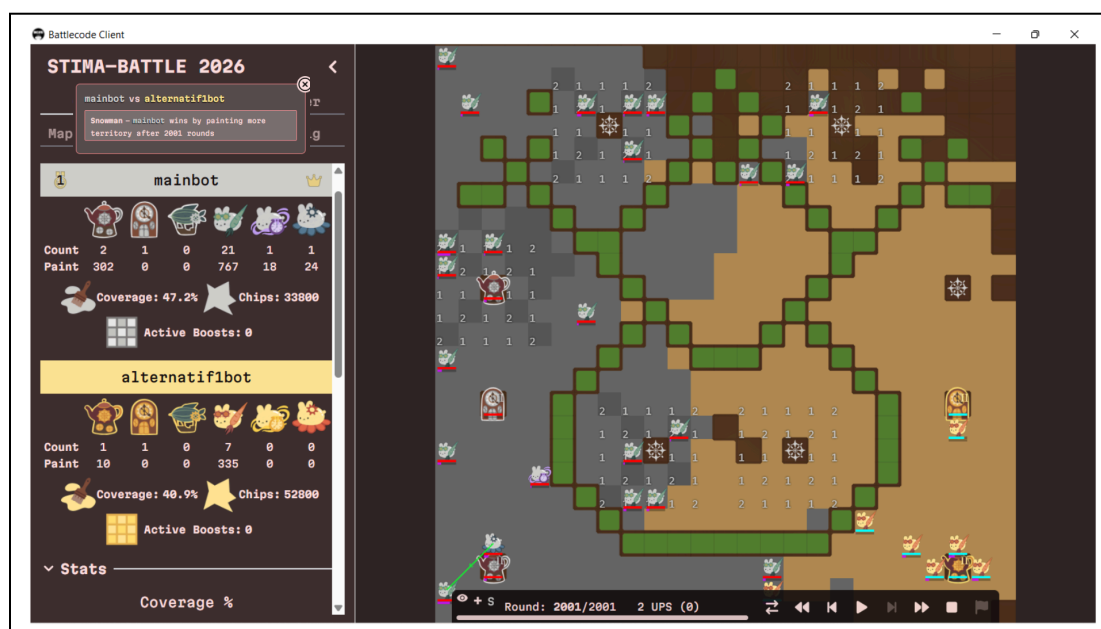
Berdasarkan tangkapan layar simulasi pada Gambar 6, Skenario Uji 4 menunjukkan kemenangan absolut dan sangat cepat bagi mainbot (Tim Abu-abu) atas examplefuncsplayer (Tim Kuning). Pertandingan ini diselesaikan secara dini hanya dalam 559 putaran (*rounds*). Pesan log pada antarmuka menegaskan bahwa

"mainbot wins by having paint majority", yang mengindikasikan bahwa mainbot berhasil memicu kondisi kemenangan mutlak dengan menguasai 70,3% luasan peta jauh sebelum batas waktu permainan berakhir.

Analisis lebih dalam pada panel statistik memperlihatkan ketimpangan performa yang sangat signifikan antara kedua agen. Mainbot sukses melakukan ekspansi masif dengan membangun 5 menara tambahan dan mengumpulkan 5.690 *chips*, serta memproduksi armada yang dominan (19 unit *Soldier*). Sebaliknya, *examplefuncsplayer* tertahan dengan hanya 1 menara awal, luasan wilayah yang sangat minim (10,7%), dan ekonomi yang stagnan (790 *chips*). Hasil uji kasus ini membuktikan bahwa strategi heuristik *Territorial Expander* dipadukan dengan taktik *Center Rush* pada mainbot sangat efektif untuk memonopoli peta secara kilat, sehingga melumpuhkan pergerakan dan pertumbuhan infrastruktur bot standar (pijakan dasar/ *baseline bot*).

4.8. Hasil Uji Kasus 5

Spesifikasi Lingkungan dan Konfigurasi Lawan: Pengujian pada skenario pertama ini dikonfigurasi untuk mengevaluasi performa Bot Utama (*Mainbot*) ketika dihadapkan dengan Bot Alternatif 1 (*Alt1*). Arena yang ditetapkan sebagai lingkungan uji coba adalah peta "snowman".



Gambar 7. Hasil Uji Kasus Skenario 05.

Berdasarkan tangkapan layar simulasi pada Gambar 7, pertandingan pada peta "Snowman" berlangsung alot hingga menyentuh batas akhir putaran permainan (ronde 2001). Sistem kembali menggunakan mekanisme *tiebreaker* (penentuan seri)

dan menetapkan mainbot (Tim Abu-abu) sebagai pemenang dengan keterangan "mainbot wins by painting more territory".

Data statistik pada panel kiri menunjukkan persaingan yang cukup ketat, di mana mainbot menguasai 47,2% wilayah, unggul tipis dibandingkan alternatifibot yang meraih 40,9%. Ketidakmampuan mainbot untuk mencapai dominasi absolut (70%) dapat diatribusikan pada karakteristik peta "Snowman" yang memiliki banyak dinding rintangan (hambatan spasial), sehingga membatasi ruang gerak algoritma penyebaran secara masif.

Lebih lanjut, analisis terhadap indikator ekonomi (*chips*) menunjukkan anomali strategi yang sangat esensial untuk dibahas. Pada akhir permainan, alternatifibot memiliki keunggulan sumber daya finansial yang sangat masif, yakni 52.800 *chips* berbanding 33.800 *chips* milik mainbot. Akan tetapi, keunggulan finansial tersebut terbukti tidak ekuivalen dengan kemenangan. Pengujian ini memberikan konklusi krusial terhadap algoritma heuristik yang dikembangkan: dalam *environment* yang mendefinisikan kemenangan *late-game* berdasarkan luasan wilayah, heuristik *Greedy* pada mainbot yang secara konstan memprioritaskan perluasan area (mewarnai ubin) terbukti lebih adaptif dan menguntungkan dibandingkan dengan agen yang hanya menimbun kekayaan tanpa bisa mengonversinya menjadi dominasi spasial.

5. POST MORTEM Analisis

Strategi Bot	Fokus Utama	Algoritma Greedy Kunci	Kekuatan Desain	Potensi Kelemahan

Bot Utama	Keseimbangan antara Ekspansi Wilayah dan Penguatan Ekonomi (Tower & Ruin Claiming)	1. Ruin Claiming. 2. Tower Construction	Memastikan basis ekonomi dan ekspansi yang cepat dan seimbang di awal permainan menggunakan sumber daya yang tersedia secara efisien.	Keputusan Ruin Claiming hanya didasarkan pada jarak terdekat yang mungkin tidak optimal secara strategis (misalnya ruin terdekat berada di posisi bertahan yang kurang penting).
Bot Alternatif 1	Dominasi Wilayah Agresif (Frontier Expansion)	1. Pain Target 2. Arah Gerak	Sangat Agresif dan cepat dalam ekspansi wilayah.	Keputusan look-ahead yang sangat lokal, menyebabkan robot terperangkap atau gagal melihat peluang yang lebih besar.
Bot Alternatif 2	Spesialisasi Sumber Daya (Resource Specialist)	1. SRP Search. 2. Upgrade Tower	Memastikan perolehan sumber daya yang stabil dan cepat untuk jangka panjang. Meminimalkan risiko kegagalan upgrade dengan memilih menara terkuat.	Fokus pada ruins terdekat dapat menyebabkan klusterisasi sumber daya. Memilih upgrade hanya berdasarkan HP mungkin mengabaikan menara yang lebih penting secara strategis (misalnya, yang berada di garis depan pertahanan).

6. Kesimpulan dan Saran

6.1. Kesimpulan

Tugas besar ini bertujuan untuk mengimplementasikan dan menganalisis penerapan Algoritma Greedy dalam pengembangan bot untuk permainan strategi,

Battlecode 2025. Pendekatan Greedy dipilih karena efisiensinya dalam mengambil keputusan optimal secara instan (optimum lokal), yang sangat krusial mengingat adanya keterbatasan sumber daya komputasi dan waktu eksekusi pada setiap giliran permainan.

Prinsip inti dari algoritma yang dirancang adalah "take what you can get now!", dengan harapan serangkaian keputusan lokal terbaik ini akan menghasilkan penguasaan wilayah peta secara maksimal (optimum global). Tiga strategi bot yang berbeda telah dikembangkan, masing-masing dengan fungsi heuristik Greedy yang unik:

- Bot Utama (Fokus Ruin Claiming & Expansion):
Menggunakan Greedy untuk memprioritaskan pembangunan menara pada ruins terdekat (meminimalkan jarak dan waktu tempuh) dan strategi Area Expansion untuk menentukan arah gerak yang memberikan potensi pewarnaan petak kosong dan perebutan petak musuh terbesar.
- Bot Alternatif 1(Frontier Expansion Specialist):
Fokus pada penguasaan wilayah secepat mungkin.
- Bot Alternatif 2(Resource Specialist):
Fokus pada keuntungan ekonomi tim.

6.2. Saran

Untuk meningkatkan bot Battlecode yang telah dikembangkan, disarankan untuk mengoptimalkan keandalan Algoritma Greedy melalui hibridisasi, di mana elemen *look-ahead* terbatas atau algoritma *pathfinding* yang lebih global (misalnya A* untuk pergerakan jarak jauh) dapat diintegrasikan guna mengatasi kelemahan *local optimum*, sambil tetap mempertahankan kecepatan Greedy untuk keputusan aksi instan di garis depan. Selain itu, penting untuk memperkuat logika pengambilan keputusan, khususnya pada Bot Alternatif 2 (Resource Specialist), dengan mengubah fungsi seleksi Upgrade Tower dari hanya berdasarkan Health Points tertinggi menjadi memprioritaskan utilitas maksimum, seperti menara di area krusial atau menara yang paling dibutuhkan tim (penghasil chips), dan pada bot ekspansi, disarankan untuk menetapkan target eksplorasi (`exploreTarget`) yang lebih agresif menuju wilayah musuh untuk mendorong dominasi peta secara lebih terarah.

7. Lampiran

7.1. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan

hasil pemikiran dan analisis mandiri.



Fazri Arrashyi Putra



Dika Pramudya Nugraha



Dzakwan Muhammad Khairan Putra Purnama

7.2. Pembagian Tugas Implementasi Bot

Tabel 1. Pembagian Tugas Implementasi Fitur BattleCode - tim *the_Brothers*

Fitur	NIM Desainer	Logika Dasar	Daftar Bot
F01 - Bot Utama	13524132	Fokus utama implementasi adalah menggabungkan ekspansi wilayah dan pembangunan menara secara adaptif. Soldier bertugas melakukan eksplorasi peta dan pewarnaan tile menggunakan pendekatan greedy untuk memilih target non-ally dengan potensi ekspansi terbesar, sekaligus mendeteksi ruin terdekat untuk dikonversi menjadi tower. Proses klaim ruin dilakukan dengan mengisi pola resource pattern hingga konstruksi menara selesai. Splasher berfungsi mempercepat dominasi area melalui serangan splash yang memaksimalkan pewarnaan tile kosong atau musuh di sekitarnya, sedangkan Mopper berperan sebagai unit pendukung yang membersihkan cat musuh dan mendistribusikan paint kepada unit sekutu yang membutuhkan. Sistem juga dilengkapi mekanisme	Splasher, Soldier, dan Mopper (opsional)

		eksplorasi terarah serta manajemen paint adaptif dengan refill otomatis ke tower terdekat untuk menjaga keberlanjutan operasi unit.	
F02 - Alternatif Bot 1	13524127	Fokus utamanya adalah memaksimalkan kontrol wilayah (tile painting) dengan algoritma greedy frontier expansion. Soldier memilih tile non-ally (prioritas empty/enemy) dengan skor heuristik tertinggi berdasarkan potensi pembukaan area cat baru, lalu bergerak ke arah yang memaksimalkan peluang paint pada turn berikutnya. Pergerakan disebar per-lane agar robot tidak menumpuk, dengan refill paint adaptif ke tower terdekat.	Soldier (utama), Splasher, & Mopper.
F03 - Alternatif Bot 2	13524145	Fokus utamanya adalah memaksimalkan ekonomi : Soldier memprioritaskan farming dengan membuat pola resource di reruntuhan (sambil menjaga stok cat), sementara Mopper berkeliling untuk menghapus cat musuh agar area farming tetap aman.	Soldier & Mopper.

7.3. Daftar Tautan Kelompok Kami

- **Link repository Github:**
https://github.com/Dikanugraha-hub/Tubes1_The-Brothers
- **Link Video Penjelasan:**
https://youtu.be/mL_01tL53Pc

8. Daftar Pustaka

- [1]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: Algoritma Greedy (Bagian 1)* [Online]. Available: 04-Algoritma-Greedy-(2026)-Bag1.pdf

- [2]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: Algoritma Greedy (Bagian 2)* [Online]. Available: 05-Algoritma-Greedy-(2026)-Bag2.pdf
- [3]. **M. Rinaldi**, *Bahan Kuliah IF2211 Strategi Algoritma: Algoritma Greedy (Bagian 3)* [Online]. Available: 06-Algoritma-Greedy-(2026)-Bag3.pdf
- [4]. **Program Studi Teknik Informatika ITB**, *Tugas Besar 1 IF2211 Strategi Algoritma Semester II tahun 2025/2026* [Online]. Available: Copy of Spesifikasi Tugas Besar 1 - IF2211 - 2025_2026.pdf